

User Authentication

Lab 1 - HTTP Basic Authentication

HTTP Basic Authentication is an HTTP authentication scheme in which the client sends a username and password with a request. The two values are joined with a colon, encoded with Base64, and placed in the Authorization header.

Lab 2 - Opaque Bearer Tokens

Token-based authentication is a broad approach in which a client first proves its identity and receives a token. The client then presents the token instead of repeatedly sending the username and password. A token may be opaque, meaning that it is only a random identifier, or structured, such as a JWT.

Lab 3 - JSON Web Tokens

A JSON Web Token is a compact claims format. A signed JWT normally contains a header, payload, and signature separated by periods. The payload can carry claims such as the subject, role, issuer, audience, issue time, and expiration time.

Lab 4 - Server-Side Sessions

Session-based authentication maintains login state for a browser or client. After successful login, the server creates a session record and gives the browser a session identifier, usually in a cookie. The browser automatically sends the cookie on later requests.

Lab 5 - OAuth 2.0 with Keycloak and Postman

OAuth 2.0 allows a client application to obtain limited access to a protected resource, normally on behalf of a resource owner. The user can authorize a client without giving the client the user's password for the resource server.

Lab 6 - OpenID Connect and Single Sign-On

Single Sign-On is an authentication experience in which a user authenticates once with a trusted identity provider and can then access multiple relying applications without entering the password again for each application.

1. Lab Overview

In this class, you will run and examine six authentication or authorization mechanisms. Four mechanisms are implemented in one Python Flask application. OAuth 2.0 and Single Sign-On are implemented with two Flask client applications and a local Keycloak identity and access management server.

Note: I can only demonstrate the main observations in one lecture, I will recommend you students to complete every implementation exercise on your own.

Lab	Technology	Primary observation
1	HTTP Basic Authentication	Reusable credentials are sent with each request.
2	Opaque bearer token	The token has no client-readable meaning and requires server lookup.
3	JWT bearer token	Signed claims can be validated without a normal token lookup.
4	Server-side session	The browser sends a cookie automatically; the server holds session state.
5	OAuth 2.0 Authorization Code with PKCE	A client obtains delegated access through an authorization server.
6	OpenID Connect and SSO	Two applications reuse the identity provider's central login session.

2. Learning Objectives

- Run a Flask application and use Postman to send authenticated HTTP requests.
- Construct and inspect an HTTP Basic Authorization header.
- Obtain, use, introspect, and revoke an opaque bearer token.
- Obtain, decode, validate, modify, and test a JWT.
- Create and terminate a browser session and inspect its cookie.
- Identify the OAuth resource owner, client, authorization server, and resource server.
- Run an Authorization Code flow with Proof Key for Code Exchange (PKCE) in Postman.
- Use OpenID Connect to authenticate through Keycloak.
- Demonstrate Single Sign-On between two independent Flask applications.
- Explain authentication failures, authorization failures, token expiration, and logout behavior.

3. Architecture and Ports

Browser / Postman

```
|
+--> Core Flask application ----- http://localhost:5000
|   |-- Basic Authentication
|   |-- Opaque bearer tokens
|   |-- JWT authentication
|   |-- Server-side sessions
|
+--> University Portal ----- http://localhost:5001
|       | +--> Keycloak auth-lab realm ----- http://localhost:8080
|       /
+--> Library Portal ----- http://localhost:5002
```

Component	Purpose	Port
Core Flask application	Labs 1-4	5000
University Portal	First OpenID Connect relying application	5001
Library Portal	Second OpenID Connect relying application	5002
Keycloak	OAuth authorization server and OpenID provider	8080

3.1 Demonstration accounts

System	Username	Password	Role
Core Flask app	alice	alice123	student
Core Flask app	admin	admin123	admin
Keycloak	alice	alice123	student
Keycloak	adminuser	admin123	student, admin
Keycloak administration	admin	admin	Keycloak administrator

Warning: These credentials are intentionally weak and exist only for a local classroom lab. Never reuse them on another system.

4. Prerequisites and Installation

4.1 Required software

- Python 3.12 or later
- Visual Studio Code or another code editor

- Postman desktop application
- Docker Desktop or a compatible Docker installation
- A modern browser with developer tools
- The supplied python-authentication-labs code folder

4.2 Create a virtual environment

```
python -m venv .venv
```

Activate the environment on Windows PowerShell:

```
.venv\Scripts\Activate.ps1
```

Activate the environment on macOS or Linux:

```
source .venv/bin/activate
```

4.3 Install dependencies

```
python -m pip install -r requirements.txt
```

4.4 Verify the codebase

```
pytest -q
```

Expected result: All five automated tests should pass. The tests cover the core Basic, opaque-token, JWT, and session flows.

4.5 Optional environment file

```
# macOS or Linux  
cp .env.example .env
```

```
# Windows PowerShell  
Copy-Item .env.example .env
```

The default values are sufficient for the local lab. In a real project, secrets must be generated securely and kept outside source control.

5. Codebase Orientation

```
python-authentication-labs/
├── core_app/
│   ├── basic_auth.py
│   ├── opaque_token_auth.py
│   ├── jwt_auth.py
│   ├── session_auth.py
│   ├── users.py
│   ├── templates/
│   └── static/
├── sso_app/
│   ├── app.py
│   ├── templates/
│   └── static/
├── keycloak/realm-export.json
├── postman/Authentication_Labs.postman_collection.json
├── tests/
├── run_core.py
├── run_sso_app_one.py
├── run_sso_app_two.py
├── docker-compose.yml
└── requirements.txt
```

File	Purpose
core_app/basic_auth.py	Parses Basic credentials and verifies the user.
core_app/opaque_token_auth.py	Issues random opaque tokens, stores them, validates them, and revokes them.
core_app/jwt_auth.py	Issues signed JWTs and validates signature and claims.
core_app/session_auth.py	Creates, reads, and clears a server-side session.
sso_app/app.py	Reusable OpenID Connect client for both SSO applications.
keycloak/realm-export.json	Preconfigures the realm, users, roles, and OAuth/OIDC clients.
postman/...collection.json	Provides ready-to-run requests for Labs 1-4.
tests/test_core_auth.py	Automated behavior checks for the core mechanisms.

6. Lab 1 - HTTP Basic Authentication

6.1 Objective

Send a username and password in an HTTP Basic Authorization header, inspect the encoded value, and compare successful and unsuccessful responses.

6.2 Start the core application

```
python run_core.py
```

Open `http://localhost:5000` and confirm that the lab home page appears.

6.3 Test without credentials

1. Open Postman and import `postman/Authentication_Labs.postman_collection.json`.
2. Open 01 Basic Authentication.
3. Send Basic Profile - No Credentials.
4. Record the HTTP status code and response body.
5. Inspect the WWW-Authenticate response header.

Expected result: The server returns 401 Unauthorized and challenges the client with a Basic authentication realm.

6.4 Test valid credentials

1. Open Basic Profile - Valid.
2. Confirm that the Authorization type is Basic Auth.
3. Use username `alice` and password `alice123`.
4. Send the request.
5. Confirm that the response contains Alice's public user information.

```
GET http://localhost:5000/basic/profile
```

6.5 Inspect the header (Optional)

1. Open the Postman Console or request headers.
2. Copy the text after "Basic" in the Authorization header.
3. Decode the value using a Base64 decoder or Python.

```
import base64
value = "YWxpY2U6YWxpY2UxMjM="
print(base64.b64decode(value).decode("utf-8"))
```

Expected observation: The result is `alice:alice123`. Base64 makes data transport-safe; it does not encrypt the password.

6.6 Failure experiment

1. Change the password to an incorrect value.
2. Send the request again.
3. Explain why the server returns 401Unauthorized rather than 403 Forbidden.

6.7 Student Take Home implementation task

- Open `core_app/basic_auth.py`.
- Locate the parsing of the Authorization header.
- Add a new GET `/basic/admin` endpoint.
- Authenticate the request using the same Basic header.
- Return 403 when the authenticated user does not have the admin role.
- Test the endpoint with alice and admin.

7. Lab 2 - Opaque Bearer Tokens

7.1 Objective

Obtain a random bearer token, use it to access a protected endpoint, examine server-side introspection, revoke the token, and verify that it can no longer be used.

7.2 Obtain a token

1. In Postman, open 02 Opaque Token.
2. Send Login and Save Token.
3. Confirm that the response contains `access_token`, `token_type`, `expires_in`, and `expires_at`.
4. Confirm that the collection test script saved the token in the `opaqueToken` variable.

```
POST http://localhost:5000/token/login
Content-Type: application/json
{
  "username": "alice",
  "password": "alice123"
}
```

7.3 Access a protected endpoint

1. Send Protected Profile.
2. Inspect the Authorization header.
3. Confirm that the response reports `server_lookup_required` as `true`.

```
Authorization: Bearer {{opaqueToken}}
```

7.4 Introspect the token

Create a Postman request using the endpoint below. The endpoint is included only to make the server-side lookup visible during the lab.

```
POST http://localhost:5000/token/introspect
Content-Type: application/json
{
  "token": "paste-the-opaque-token"
}
```

Expected result: A valid stored token returns `active: true`, its username, and expiration time. The token itself does not reveal those values.

7.5 Revoke and reuse

Send Revoke Token and send Protected Profile again without changing the token.

Confirm that the same token now returns 401 and send the introspection request again and confirm `active: false`.

7.6 Student Take Home implementation task

- Open `core_app/opaque_token_auth.py`.
- Add a token name or device label to `TokenRecord`. Accept the label during login and return it from the introspection endpoint.

8. Lab 3 - JSON Web Tokens

8.1 Objective

Obtain a signed JWT, inspect its claims, use it for authentication and role-based authorization, and observe how signature and expiration validation detect invalid tokens.

8.2 Obtain and use a JWT

1. In Postman, open 03 JWT.
2. Send Login and Save JWT using alice/alice123.
3. Confirm that the jwtToken collection variable is populated.
4. Send JWT Profile.
5. Identify sub, role, iss, aud, iat, nbf, and exp in the returned claims.

```
POST http://localhost:5000/jwt/login
Content-Type: application/json
{
  "username": "alice",
  "password": "alice123"
}
```

8.3 Decode the JWT (Optional)

Copy the JWT and separate it at the period characters. Decode the first two parts using a JWT inspection tool or a short Python program. Do not treat the decoded payload as trusted until signature validation succeeds.

```
import base64, json
def decode_part(value):
    value += "=" * (-len(value) % 4)
    return json.loads(base64.urlsafe_b64decode(value))

header, payload, signature = TOKEN.split(".")
print(decode_part(header))
print(decode_part(payload))
```

8.4 Test authorization

1. Use Alice's JWT with GET /jwt/admin.
2. Record the 403 response.
3. Log in with admin/admin123.
4. Use the new administrator JWT with GET /jwt/admin.
5. Record the successful response.

Interpretation: Alice is authenticated, so the token is valid, but she lacks the required role. Therefore the correct result is 403 Forbidden.

8.5 Tampering experiment

1. Decode Alice's JWT payload.

2. Change role from student to admin.
3. Re-encode the modified payload without creating a valid new signature.
4. Submit the modified token to `/jwt/admin`.
5. Record the validation failure.

8.6 Expiration experiment

1. Open `.env` or temporarily change `JWT_EXPIRATION_SECONDS` to 15.
2. Restart the core Flask application.
3. Obtain a new JWT.
4. Use it immediately and confirm success.
5. Wait more than 15 seconds and reuse it.
6. Confirm that the server returns a token-expired error.
7. Restore the value to 300 seconds after the experiment.

8.7 Student Take Home implementation task

- Add a permissions claim containing a list such as `["profile:read"]`.
- Create `GET /jwt/permissions`.
- Require the `profile:read` permission.
- Return 403 when the claim is missing.

9. Lab 4 - Server-Side Sessions

9.1 Objective

Create a browser login session, inspect the session identifier cookie, verify automatic cookie transmission, and compare local session logout with bearer-token behavior.

9.2 Create a browser session

1. Open `http://localhost:5000/session/login` in a browser.
2. Log in with `alice` and `alice123`.
3. Confirm that the profile page appears.
4. Refresh the page and observe that no password is requested.

9.3 Inspect the cookie

1. Open browser developer tools.
2. Open Application or Storage, then Cookies, then `http://localhost:5000`.
3. Find `auth_lab_session`.
4. Record the `HttpOnly` and `SameSite` attributes.
5. Open the Network tab, refresh the page, and inspect the Cookie request header.

Expected observation: The browser sends the cookie automatically. The cookie contains a session identifier; the authenticated session data is stored in the server-side cache.

9.4 Delete and recreate the cookie

1. Delete `auth_lab_session` in developer tools.
2. Refresh `/session/profile`.
3. Confirm that the application redirects to login.
4. Log in again and confirm that a new cookie value is created.

9.5 Log out

1. Click Log out.
2. Attempt to return to `/session/profile`.
3. Confirm that the server no longer recognizes an authenticated session.

9.6 Postman session experiment

1. Send Session Login in the imported collection.
2. Open Postman's cookie manager and find the cookie for localhost.
3. Send Session Profile.
4. Delete the cookie and send Session Profile again.
5. Explain why Postman can behave like a browser cookie client.

9.7 Student implementation task

- Store `login_time` in the server-side session.
- Display the value on the profile page.
- Add a session idle timeout or explain how one would be configured.

10. Lab 5 - OAuth 2.0 with Keycloak and Postman

10.1 Objective

Run a local authorization server, use the Authorization Code flow with PKCE, obtain OAuth and OpenID Connect tokens, and identify the actors and security checks in the flow.

10.2 Start Keycloak

```
docker compose up -d
```

Wait until <http://localhost:8080> is available. The supplied realm file automatically creates the auth-lab realm, users, roles, and clients.

```
docker compose logs -f keycloak
```

Keycloak administration: Open <http://localhost:8080/admin> and sign in with admin/admin. Select the auth-lab realm before examining clients or users.

10.3 Identify the OAuth actors

OAuth role	Lab component
Resource owner	The user Alice
Client	Postman
Authorization server	Keycloak
Resource server	A protected API that accepts the resulting access token

10.4 Configure Postman

Setting	Value
Grant type	Authorization Code with PKCE
Callback URL	https://oauth.pstmn.io/v1/callback
Authorization URL	http://localhost:8080/realms/auth-lab/protocol/openid-connect/auth
Access Token URL	http://localhost:8080/realms/auth-lab/protocol/openid-connect/token
Client ID	postman-client
Client secret	Leave empty; this is a public client
Scope	openid profile email
Code challenge method	SHA-256 / S256

10.5 Run the flow

1. In Postman, create or open a request.

2. Select Authorization and OAuth 2.0.
3. Enter the values in the preceding table.
4. Click Get New Access Token.
5. When Keycloak opens, sign in with alice and alice123.
6. Complete any consent screen if it appears.
7. Allow Postman to complete the callback.
8. Select Use Token.

Inspect the token response fields.

Postman OAuth settings

Use these values:

```
Grant Type:
Authorization Code (With PKCE)

Callback URL:
https://oauth.pstmn.io/v1/browser-callback

Auth URL:
http://localhost:8080/realms/auth-lab/protocol/openid-connect/auth

Access Token URL:
http://localhost:8080/realms/auth-lab/protocol/openid-connect/token

Client ID:
postman-client

Client Secret:
Leave blank

Scope:
openid profile email roles

Code Challenge Method:
SHA-256

Authorize using browser:
Enabled
```

10.6 Inspect the browser flow

- The browser is redirected to Keycloak rather than giving the password to Postman directly.
- The authorization request contains client_id, redirect_uri, response_type=code, scope, state, code_challenge, and code_challenge_method.
- The callback contains an authorization code rather than the user's password.
- Postman exchanges the code and PKCE verifier at the token endpoint.
- The access token represents authorized access; the ID Token represents authentication information because the openid scope was requested.

10.7 Student Take Home Implementation task

1. Remove openid from the requested scope and repeat the request. Compare the token result.
2. Enter an incorrect Client ID and record the error.
3. Use an unregistered callback URL and record the error.

11. Lab 6 - OpenID Connect and Single Sign-On

11.1 Objective

Authenticate two independent Flask applications through one Keycloak realm and demonstrate that the second application can reuse the identity provider's authenticated browser session.

11.2 Start both applications

Keep Keycloak running. Open two terminals with the same virtual environment activated.

```
# Terminal 1
python run_sso_app_one.py

# Terminal 2
python run_sso_app_two.py
```

11.3 Authenticate Application One

1. Open `http://localhost:5001`.
2. Click Log in with Keycloak.
3. Sign in with alice and alice123.
4. Confirm that the University Portal displays the preferred username, name, email, and subject identifier.
5. Inspect browser cookies for localhost and locate `sso_app_one_session`.

11.4 Demonstrate SSO

1. Open `http://localhost:5002` in the same browser profile.
2. Click Log in with Keycloak.
3. Observe whether Keycloak asks for the password again.
4. Confirm that the Library Portal creates its own local session.
5. Inspect cookies and locate `sso_app_two_session`.

Critical observation: The two applications do not share one Flask cookie. They have separate local sessions. The password is not requested again because Keycloak has a separate central authentication session.

11.5 Local logout versus single sign-out

1. In Application One, use Local logout.
2. Return to Application One and click login again.
3. Observe that Keycloak may immediately log the user back in because the Keycloak session still exists.
4. Use Single sign-out through Keycloak.
5. Return to the other application and test whether a fresh login is required after its local session is cleared or expires.
6. Explain the difference among local logout, identity-provider logout, and complete cross-application logout behavior.

11.6 Inspect the OIDC code

- Open `sso_app/app.py`.
- Locate `server_metadata_url`. This uses OpenID Connect discovery.
- Locate `scope: openid profile email`.
- Locate `code_challenge_method: S256`.
- Locate `authorize_redirect` and `authorize_access_token`.
- Locate the storage of `userinfo` and `id_token` in the local Flask session.
- Locate `logout_redirect` for identity-provider logout.

11.7 Student Take Home implementation task

- Add a page that displays only the non-sensitive claim names received from Keycloak.
- Add a role check for the admin realm role.
- Display an authorization failure message for Alice.
- Test the page with `admin/admin123`.
- You may face CORS Header error in the browser, so make sure to resolve that.

12. Security Limitations

The code intentionally exposes implementation details for learning. It is not a production security template. A production system would require additional controls, including:

- HTTPS/TLS for every credential, cookie, code, and token exchange.
- Persistent user, token, and session data stores with access controls and backups.
- Production-grade secret storage, key generation, rotation, and separation.
- A production WSGI server and hardened deployment configuration.
- CSRF tokens for state-changing browser requests.
- Rate limiting, account throttling, audit logging, and security monitoring.
- Secure password policy and modern password-hashing configuration.
- Refresh-token rotation and token-reuse detection where refresh tokens are used.
- Robust JWT key management, asymmetric signing where appropriate, and JWKS rotation.
- Explicit scope, role, audience, and resource authorization policy.
- Session expiration, idle timeout, invalidation, and concurrent-session management.
- Dependency monitoring and timely security updates.
- Threat modeling, security testing, and independent review.

13. References

- Flask Documentation: <https://flask.palletsprojects.com/en/stable/>
- Flask session quickstart: <https://flask.palletsprojects.com/en/stable/quickstart/#sessions>
- PyJWT Documentation: <https://pyjwt.readthedocs.io/en/stable/>
- Authlib Flask Client Documentation: <https://docs.authlib.org/en/latest/oauth2/client/web/flask.html>
- Keycloak Documentation: <https://www.keycloak.org/documentation>
- Keycloak OpenID Connect Guide: <https://www.keycloak.org/securing-apps/oidc-layers>
- IETF RFC 7617, Basic Authentication: <https://datatracker.ietf.org/doc/html/rfc7617>
- IETF RFC 6750, Bearer Token Usage: <https://datatracker.ietf.org/doc/html/rfc6750>
- IETF RFC 7519, JSON Web Token: <https://datatracker.ietf.org/doc/html/rfc7519>
- IETF RFC 6749, OAuth 2.0: <https://datatracker.ietf.org/doc/html/rfc6749>
- OpenID Connect Core: https://openid.net/specs/openid-connect-core-1_0.html
- Python secrets module: <https://docs.python.org/3/library/secrets.html>